C.5 Weighted Finite-state Transducers

Definition C.13. *A (rational-)**weighted finite-state transducer** (WFST) is the tuple* $(\Sigma, \Xi, Q, \delta, \lambda, \rho)$ *where* Σ *and* Ξ *are the input and output alphabets, respectively,* Q *is a finite set of states,* $\delta \subseteq Q \times \Sigma_\varepsilon \times \Xi_\varepsilon \times \mathbb{Q}_{\geq 0} \times Q$ *is a finite set of weighted transitions where we write transitions* $(q, y, x, w, q') \in \delta$ *as* $q \xrightarrow{y:x/w} q'$*, and* $\lambda, \rho \colon Q \to \mathbb{Q}_{\geq 0}$ *are functions that assign each state its initial and final weight, respectively.*

WFSTs are thus a special class of (weighted) finite-state automata that operate over two alphabets. Just like PFSAs are a generalization of unweighted FSAs, WFSAs represent a more general class of machines than unweighted FSTs, such as the ones used in the definition of regular reducibility (cf. Def. 3.1). The weighted versions will prove useful when talking about various aspects of equivalence with regular reducibility. As a special case of weighted finite-state automata, WFSTs compute weights of their inputs—in this case, weights of *pairs* of strings (the inputs and their outputs):

$$\mathcal{T}(\boldsymbol{y}, \boldsymbol{x}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{\pi} \in \Pi(\mathcal{T}, (\boldsymbol{y}, \boldsymbol{x}))} \mathrm{w}(\boldsymbol{\pi}). \tag{36}$$

Of interest are also marginal sums of the inputs, i.e.,

$$\mathcal{T}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{x} \in \Xi^*} \mathcal{T}(\boldsymbol{y}, \boldsymbol{x}). \tag{37}$$

C.5.1 Operations on WFSTs

We will use WFSTs as building blocks in the exposition of regular reducibility (cf. §3.2). In this subsection, we outline some operations on the computational models that will be particularly useful.

Composition. The **composition** of two WFSTs results in a WFST that, similarly to function composition, maps the strings with the first WFST, passes them to the second one, and returns the (weight of the) output of the second transducer (Pereira and Riley, 1997). More formally, given the WFSTs $\mathcal{T}_1 = (\Sigma, \Upsilon, Q, \delta, \lambda, \rho)$ and $\mathcal{T}_2 = (\Upsilon, \Xi, Q, \delta, \lambda, \rho)$, their composition $\mathcal{T}_2 \circ \mathcal{T}_1$ computes

$$(\mathcal{T}_2 \circ \mathcal{T}_1)(\boldsymbol{y}, \boldsymbol{x}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{z} \in \Upsilon^*} \mathcal{T}_2(\boldsymbol{z}, \boldsymbol{y}) \cdot \mathcal{T}_1(\boldsymbol{x}, \boldsymbol{z}). \tag{38}$$

Intuitively, Eq. (38) computes the weight of all the possible ways of mapping $\boldsymbol{y}$ to $\boldsymbol{x}$ by first mapping $\boldsymbol{y}$ to some $\boldsymbol{z} \in \Upsilon^*$ and then mapping that $\boldsymbol{z}$ into $\boldsymbol{x}$. The WFST $(\mathcal{T}_2 \circ \mathcal{T}_1)$ can be computed from $\mathcal{T}_1$ and $\mathcal{T}_2$ in time $\mathcal{O}((|Q_1| + |\delta_1|)(|Q_2| + |\delta_2|))$ (Mohri, 2009).

Projecting a WFST. Any WFST can be **projected** onto an (input or output) weighted finite-state *automaton* (WFSA)[24] that computes the cumulative weights of individual input or output strings (Mohri et al., 2008). Given a WFST $\mathcal{T} = (\Sigma, \Xi, Q, \delta, \lambda, \rho)$, its input projection is the automaton $\mathcal{A}_\mathcal{I} = (\Sigma, Q, \delta_\mathcal{I}, \lambda, \rho)$ that computes

$$\mathcal{A}_\mathcal{I}(\boldsymbol{y}) \stackrel{\text{def}}{=} \sum_{\boldsymbol{x} \in \Xi^*} \mathcal{T}(\boldsymbol{y}, \boldsymbol{x}). \tag{39}$$

The output projection automaton is defined analogously. While it might not be immediately obvious that Eq. (39) represents the computations of a WFSA, $\mathcal{A}_\mathcal{I}$ can be easily constructed by ignoring the output labels on the transitions (and additively merging any transitions that become identical after the removal of the output labels). That is:

$$\delta_\mathcal{I} = \left\{ q \xrightarrow{y/\sum_{q \xrightarrow{y:x/w} q' \in \delta} w} q' \mid q, q' \in Q, y \in \Sigma_\varepsilon \right\}. \tag{40}$$

[24] A weighted finite-state automaton is a generalization of a probabilistic finite-state automaton where the weights do not have to form probability distributions. We also extend the definition of WFSAs to allow ε-transitions, i.e. with symbols from Σ_ε. Note that every such WFSA can be converted into a weakly equivalent WFSA without ε-transitions.

Lifting a PFSA. **Lifting** transforms a given PFSA into a trivial WFST that implements the identity function $\boldsymbol{y} \mapsto \boldsymbol{y}$. Concretely, given the PFSA $\mathcal{A} = (\Sigma, Q, \lambda, \rho, \delta)$, its lifted WFST $\mathcal{T}_{\mathcal{A}} = (\Sigma, \Sigma, Q, \delta_{\mathcal{T}}, \lambda, \rho)$ defines the transitions

$$\delta_{\mathcal{T}} \stackrel{\text{def}}{=} \left\{ q \xrightarrow{y:y/w} q' \mid q \xrightarrow{y/w} q' \in \delta \right\}. \tag{41}$$

It is easy to see that $\mathcal{T}_{\mathcal{A}}$ computes

$$\mathcal{T}_{\mathcal{A}}\left(\boldsymbol{y}, \boldsymbol{y}'\right) = \mathbb{1}\left\{\boldsymbol{y} = \boldsymbol{y}'\right\} \mathcal{A}\left(\boldsymbol{y}\right). \tag{42}$$

Lifting is useful when one wants to compose a PFSA with a WFST, as is the case when discussing regular reducibility.

Inverting a WFST. The **inverted** WFST of some WFST $\mathcal{T} = (\Sigma, \Xi, Q, \delta, \lambda, \rho)$ is the WSFT $\mathcal{T}^{-1} = \left(\Xi, \Sigma, Q, \delta^{-1}, \lambda, \rho\right)$ that maps the outputs of $\mathcal{T}$ to their inputs. δ^{-1} is defined as

$$\delta^{-1} \stackrel{\text{def}}{=} \left\{ q \xrightarrow{x:y/w} q \mid q \xrightarrow{y:x/w} q \in \delta \right\}, \tag{43}$$

i.e., it is composed of transitions from δ with their input–output labels flipped.

D Separation of Alphabets

In our analysis, we use a different alphabet for the chain of thought (Γ) than for the final output (Σ) to clarify the distinction between output that encodes automata configurations vs the output that constitutes the weighted output language of that automaton. However, we can also define a regular function for the case where $\Gamma = \Sigma$. For this, we choose a specific symbol in Σ, e.g., X, whose first occurrence signifies the boundary between the chain of thought and the language output. Then, ϕ can be defined as:

$$\phi(\boldsymbol{y}\, X\, \boldsymbol{z}) \stackrel{\text{def}}{=} \boldsymbol{z} \tag{44}$$

where $\boldsymbol{y} \in \Sigma^* \backslash \{X\}$, and $\boldsymbol{z} \in \Sigma^*$. This can be easily modeled by the following simple two-state transducer:

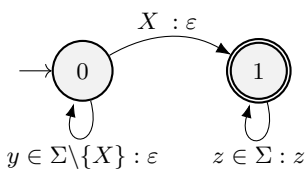

Figure 4: A simple WFST $\mathcal{T}$ implementing ϕ.

Note that in this case, the chain of thought has to be confined in its entirety as one string, with the entire final output coming afterward. This simply means that the simulated Turing machine first performs all the computations it needs on the working tape, and then writes the final string to the output tape in real time. The proof of Thm. 4.4 can easily be adapted to work in this setting by only tagging the ε symbols in Γ, e.g. by symbols a and b and defining ϕ to remove them until the occurrence of another symbol, e.g. c.

E Proofs: Regular Reducibility

E.1 Nondeterminism in PFSAs

Theorem 3.1. *Let $\mathcal{A} = (\Sigma, Q, \delta, \lambda, \rho)$ be a PFSA. Then, there exists a* deterministic *PFSA $\mathcal{A}'$ over the alphabet $\Sigma \times Q$ and with the state space $\Sigma \times Q$ that is regularly reducible to $\mathcal{A}$.*

Proof. Given the PFSA $\mathcal{A} = (\Sigma, Q, \lambda, \rho, \delta)$, we want to show the existence of a *deterministic* PFSA $\mathcal{A}'$ over the alphabet $\Sigma \times Q$ that is regularly reducible to $\mathcal{A}$. We construct the deterministic PFSA $\mathcal{A}' = \left(\Sigma \times Q, \Sigma \times Q, \delta', \lambda', \rho'\right)$ as follows.

$$\delta' \stackrel{\text{def}}{=} \left\{ (y, q) \xrightarrow{(y',q')/w} \left(y', q'\right) \mid q \xrightarrow{y'/w} q' \in \delta, y \in \Sigma \right\} \tag{45a}$$

$$\lambda'(y, q) \stackrel{\text{def}}{=} \frac{\lambda(q)}{|\Sigma|}, \quad \forall y \in \Sigma, q \in Q \tag{45b}$$

$$\rho'(y, q) \stackrel{\text{def}}{=} \rho(q), \quad \forall y \in \Sigma, q \in Q \tag{45c}$$

We first prove that $\mathcal{A}'$ is probabilistic and deterministic.

- **Probabilistic**: We compute

$$\sum_{(y,q) \in \Sigma \times Q} \lambda'(y, q) = \sum_{(y,q) \in \Sigma \times Q} \frac{\lambda(q)}{|\Sigma|} \tag{46a}$$

$$= \sum_{y \in \Sigma} \sum_{q \in Q} \frac{\lambda(q)}{|\Sigma|} \tag{46b}$$

$$= \sum_{q \in Q} \lambda(q) \tag{46c}$$

$$= 1 \tag{46d}$$

and, for any $(y, q) \in \Sigma \times Q$,

$$\sum_{(y,q) \xrightarrow{(y',q')/w} (y',q') \in \delta'} w + \rho'(y, q) = \sum_{(y,q) \xrightarrow{(y',q')/w} (y',q') \in \delta'} w + \rho(q) \tag{47a}$$

$$= \sum_{q \xrightarrow{w/y'} y' \in \delta} w + \rho(q) \tag{47b}$$

$$= 1 \tag{47c}$$

- **Deterministic**: Let $(y, q) \in \Sigma \times Q$ be a state of $\mathcal{A}'$ and (y', q') a symbol in its alphabet. By definition of δ', (y', q') uniquely determines the target state, which is identical to the symbol—(y', q').

We now show that $\mathcal{A}'$ is indeed regularly reducible to $\mathcal{A}$. The alphabet $\Sigma \times Q$ clearly satisfies the Σ-augmentation condition. We define the regular function $\phi(y, q) \stackrel{\text{def}}{=} y$, an instance of the general function described in §3.2. We will show that $\mathcal{A}'$ is weakly equivalent to $\mathcal{A} \circ \phi$, or, equivalently, that $\mathcal{A}' \circ \phi^{-1}$ is weakly equivalent to $\mathcal{A}$.

Since $\mathcal{A}'$ is an *acceptor* of strings in $(\Sigma \times Q)^*$, we first transform it into weighted finite-state *transducer* $\mathcal{T}_{\mathcal{A}}$ implementing the mapping[25]

$$\mathcal{T}_{\mathcal{A}'}\left(\boldsymbol{x}, \boldsymbol{x}'\right) \stackrel{\text{def}}{=} \mathbb{1}\left\{\boldsymbol{x} = \boldsymbol{x}'\right\} \mathcal{A}(\boldsymbol{x}) \tag{48}$$

for $\boldsymbol{x}, \boldsymbol{x}' \in \Delta^*$. This is a simple WFST with transitions $\delta_{\mathcal{T}_{\mathcal{A}'}} \stackrel{\text{def}}{=} \left\{ q \xrightarrow{(y,q):(y,q)/w} q' \mid q \xrightarrow{(y,q)/w} q' \in \delta' \right\}$. $\mathcal{T}_{\mathcal{A}'}$ can then be composed with the weighed version of the FST implementing ϕ^{-1}, $\mathcal{T}_{\phi^{-1}}$:

$$\mathcal{T}_{\phi^{-1}}(\boldsymbol{y}, \boldsymbol{x}) \stackrel{\text{def}}{=} \mathbb{1}\left\{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})\right\}. \tag{49}$$

[25] See App. C.5 for a definition of weighted finite-state transducers and the operations on them.